

Import libraries

```
# %%  
# Prototype: CNN + Grad-CAM on CBIS-DDSM subset  
  
import os  
import sys  
import cv2  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
  
import torch  
import torch.nn as nn  
from torch.utils.data import DataLoader  
from torchvision import models  
from torchvision import transforms  
  
# Make project root importable  
ROOT_DIR = os.path.abspath(os.path.join(".."))  
if ROOT_DIR not in sys.path:  
    sys.path.append(ROOT_DIR)  
  
# from utils.dataset import CBISDicomDataset # you'll create/adjust this  
from utils.dataset import CBISDataset  
  
from utils.transforms import get_preprocessing_transforms  
from utils.gradcam import GradCAM  
  
device = "cuda" if torch.cuda.is_available() else "cpu"  
device  
ROOT_DIR  
  
'/Users/gibionmakiwa/MyDocuments/610MtSundanceBayW/University_of_London_Computer_Science_BS
```

Point to archive

```
RAW_ROOT = os.path.join(ROOT_DIR, "data", "raw", "archive")  
  
CSV_DIR = os.path.join(RAW_ROOT, "csv")  
JPEG_DIR = os.path.join(RAW_ROOT, "jpeg")  
  
meta_path = os.path.join(CSV_DIR, "meta.csv")  
dicom_info_path = os.path.join(CSV_DIR, "dicom_info.csv")  
  
meta = pd.read_csv(meta_path)  
meta.head()
```

	SeriesInstanceUID \
0	1.3.6.1.4.1.9590.100.1.2.117041576511324414842...
1	1.3.6.1.4.1.9590.100.1.2.438738396107617880132...
2	1.3.6.1.4.1.9590.100.1.2.767416741131676463382...
3	1.3.6.1.4.1.9590.100.1.2.296931352612305599800...
4	1.3.6.1.4.1.9590.100.1.2.436657670120353100077...

	StudyInstanceUID	Modality \
0	1.3.6.1.4.1.9590.100.1.2.229361142710768138411...	MG
1	1.3.6.1.4.1.9590.100.1.2.195593486612988388325...	MG
2	1.3.6.1.4.1.9590.100.1.2.257901172612530623323...	MG
3	1.3.6.1.4.1.9590.100.1.2.109468616710242115222...	MG
4	1.3.6.1.4.1.9590.100.1.2.380627129513562450304...	MG

	SeriesDescription	BodyPartExamined	SeriesNumber	Collection	Visibility \
0	ROI mask images	BREAST	1	CBIS-DDSM	1
1	ROI mask images	BREAST	1	CBIS-DDSM	1
2	ROI mask images	BREAST	1	CBIS-DDSM	1
3	ROI mask images	BREAST	1	CBIS-DDSM	1
4	ROI mask images	BREAST	1	CBIS-DDSM	1

	ImageCount
0	2
1	2
2	2
3	2
4	2

```
dicom_info = pd.read_csv(dicom_info_path)
dicom_info.head()
```

	file_path \
0	CBIS-DDSM/dicom/1.3.6.1.4.1.9590.100.1.2.12930...
1	CBIS-DDSM/dicom/1.3.6.1.4.1.9590.100.1.2.24838...
2	CBIS-DDSM/dicom/1.3.6.1.4.1.9590.100.1.2.26721...
3	CBIS-DDSM/dicom/1.3.6.1.4.1.9590.100.1.2.38118...
4	CBIS-DDSM/dicom/1.3.6.1.4.1.9590.100.1.2.38118...

	image_path	AccessionNumber \
0	CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.129308...	NaN
1	CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.248386...	NaN
2	CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.267213...	NaN
3	CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.381187...	NaN
4	CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.381187...	NaN

	BitsAllocated	BitsStored	BodyPartExamined	Columns	ContentDate \
0	16	16	BREAST	351	20160426

1	16	16	BREAST	3526	20160426
2	16	16	BREAST	1546	20160503
3	16	16	BREAST	97	20160503
4	8	8	Left Breast	3104	20160503

	ContentTime	ConversionType	...	\
0	131732.685	WSD	...	
1	143829.101	WSD	...	
2	111956.298	WSD	...	
3	115347.770	WSD	...	
4	115347.770	WSD	...	

	SecondaryCaptureDeviceManufacturerModelName	SeriesDescription	\
0	MATLAB	cropped images	
1	MATLAB	full mammogram images	
2	MATLAB	full mammogram images	
3	MATLAB	cropped images	
4	MATLAB	NaN	

	SeriesInstanceUID	SeriesNumber	\
0	1.3.6.1.4.1.9590.100.1.2.129308726812851964007...	1	
1	1.3.6.1.4.1.9590.100.1.2.248386742010678582309...	1	
2	1.3.6.1.4.1.9590.100.1.2.267213171011171858918...	1	
3	1.3.6.1.4.1.9590.100.1.2.381187369611524586537...	1	
4	1.3.6.1.4.1.9590.100.1.2.381187369611524586537...	1	

	SmallestImagePixelValue	SpecificCharacterSet	StudyDate	StudyID	\
0	23078	ISO_IR 100	20160720.0	DDSM	
1	0	ISO_IR 100	20160720.0	DDSM	
2	0	ISO_IR 100	20160807.0	DDSM	
3	32298	ISO_IR 100	20170829.0	DDSM	
4	0	ISO_IR 100	NaN	DDSM	

	StudyInstanceUID	StudyTime
0	1.3.6.1.4.1.9590.100.1.2.271867287611061855725...	214951.0
1	1.3.6.1.4.1.9590.100.1.2.161516517311681906612...	193426.0
2	1.3.6.1.4.1.9590.100.1.2.291043622711253836701...	161814.0
3	1.3.6.1.4.1.9590.100.1.2.335006093711888937440...	180109.0
4	1.3.6.1.4.1.9590.100.1.2.335006093711888937440...	NaN

[5 rows x 38 columns]

check image types in dataset

dicom_info.SeriesDescription.unique()

array(['cropped images', 'full mammogram images', nan, 'ROI mask images'],
dtype=object)

```

# check image path in dataset
# cropped images
cropped_images = dicom_info[dicom_info.SeriesDescription=='cropped images'].image_path
cropped_images.head(5)

0      CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.129308...
3      CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.381187...
6      CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.153339...
7      CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.178994...
10     CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.411833...
Name: image_path, dtype: object

#full mammogram images
full_mammo = dicom_info[dicom_info.SeriesDescription=='full mammogram images'].image_path
full_mammo.head(5)

1      CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.248386...
2      CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.267213...
11     CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.210396...
12     CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.749566...
15     CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.987658...
Name: image_path, dtype: object

# ROI images
roi_img = dicom_info[dicom_info.SeriesDescription=='ROI mask images'].image_path
roi_img.head(5)

5      CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.153339...
8      CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.178994...
9      CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.411833...
14     CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.236373...
20     CBIS-DDSM/jpeg/1.3.6.1.4.1.9590.100.1.2.357008...
Name: image_path, dtype: object

# change directory path of images
cropped_images = cropped_images.replace('CBIS-DDSM/jpeg', JPEG_DIR, regex=True)
full_mammo = full_mammo.replace('CBIS-DDSM/jpeg', JPEG_DIR, regex=True)
roi_img = roi_img.replace('CBIS-DDSM/jpeg', JPEG_DIR, regex=True)

# view new paths
print('Cropped Images paths:\n')
print(cropped_images.iloc[0])
print('Full mammo Images paths:\n')
print(full_mammo.iloc[0])
print('ROI Mask Images paths:\n')
print(roi_img.iloc[0])

Cropped Images paths:

```

/Users/gibionmakiwa/MyDocuments/610MtSundanceBayW/University_of_London_Computer_Science_BSc/
Full mammo Images paths:

/Users/gibionmakiwa/MyDocuments/610MtSundanceBayW/University_of_London_Computer_Science_BSc/
ROI Mask Images paths:

/Users/gibionmakiwa/MyDocuments/610MtSundanceBayW/University_of_London_Computer_Science_BSc/

load the mass dataset

mass_train = pd.read_csv(os.path.join(CSV_DIR, "mass_case_description_train_set.csv"))

mass_test = pd.read_csv(os.path.join(CSV_DIR, "mass_case_description_test_set.csv"))

mass_train.head()

	patient_id	breast_density	left or right	breast image view	abnormality id	\
0	P_00001	3	LEFT	CC	1	
1	P_00001	3	LEFT	MLO	1	
2	P_00004	3	LEFT	CC	1	
3	P_00004	3	LEFT	MLO	1	
4	P_00004	3	RIGHT	MLO	1	

	abnormality type	mass shape	mass margins	\
0	mass	IRREGULAR-ARCHITECTURAL_DISTORTION	SPICULATED	
1	mass	IRREGULAR-ARCHITECTURAL_DISTORTION	SPICULATED	
2	mass	ARCHITECTURAL_DISTORTION	ILL_DEFINED	
3	mass	ARCHITECTURAL_DISTORTION	ILL_DEFINED	
4	mass	OVAL	CIRCUMSCRIBED	

	assessment	pathology	subtlety	\
0	4	MALIGNANT	4	
1	4	MALIGNANT	4	
2	4	BENIGN	3	
3	4	BENIGN	3	
4	4	BENIGN	5	

	image file path	\
0	Mass-Training_P_00001_LEFT_CC/1.3.6.1.4.1.9590...	
1	Mass-Training_P_00001_LEFT_MLO/1.3.6.1.4.1.959...	
2	Mass-Training_P_00004_LEFT_CC/1.3.6.1.4.1.9590...	
3	Mass-Training_P_00004_LEFT_MLO/1.3.6.1.4.1.959...	
4	Mass-Training_P_00004_RIGHT_MLO/1.3.6.1.4.1.95...	

	cropped image file path	\
0	Mass-Training_P_00001_LEFT_CC_1/1.3.6.1.4.1.95...	
1	Mass-Training_P_00001_LEFT_MLO_1/1.3.6.1.4.1.9...	
2	Mass-Training_P_00004_LEFT_CC_1/1.3.6.1.4.1.95...	
3	Mass-Training_P_00004_LEFT_MLO_1/1.3.6.1.4.1.9...	

```

4 Mass-Training_P_00004_RIGHT_MLO_1/1.3.6.1.4.1....

                                ROI mask file path
0 Mass-Training_P_00001_LEFT_CC_1/1.3.6.1.4.1.95...
1 Mass-Training_P_00001_LEFT_MLO_1/1.3.6.1.4.1.9...
2 Mass-Training_P_00004_LEFT_CC_1/1.3.6.1.4.1.95...
3 Mass-Training_P_00004_LEFT_MLO_1/1.3.6.1.4.1.9...
4 Mass-Training_P_00004_RIGHT_MLO_1/1.3.6.1.4.1....

print(len(cropped_images), len(roi_img), len(mass_train))

3567 3247 1318

```

Convert to arrays for training

```

# Fix column names (your CSV has spaces)
mass_train = mass_train.rename(columns={
    "image file path": "img_path",
    "cropped image file path": "cropped_path",
    "ROI mask file path": "mask_path"
})

mass_test = mass_test.rename(columns={
    "image file path": "img_path",
    "cropped image file path": "cropped_path",
    "ROI mask file path": "mask_path"
})

# Ensure both train and test have the same columns
for df in [mass_train, mass_test]:
    for col in ["img_path", "cropped_path", "mask_path"]:
        if col not in df.columns:
            df[col] = np.nan

# Combine train + test
full_mass = pd.concat([mass_train, mass_test], axis=0).reset_index(drop=True)
#print(full_mass.columns.tolist())
print(full_mass[["img_path", "cropped_path", "mask_path"]].head())

full_mass.to_csv(CSV_DIR +
    "/full_mass.csv",
    index=False
)

print(type(full_mass))
print(full_mass.head())

```

```

img_path \
0 Mass-Training_P_00001_LEFT_CC/1.3.6.1.4.1.9590...
1 Mass-Training_P_00001_LEFT_MLO/1.3.6.1.4.1.959...
2 Mass-Training_P_00004_LEFT_CC/1.3.6.1.4.1.9590...
3 Mass-Training_P_00004_LEFT_MLO/1.3.6.1.4.1.959...
4 Mass-Training_P_00004_RIGHT_MLO/1.3.6.1.4.1.95...

```

```

cropped_path \
0 Mass-Training_P_00001_LEFT_CC_1/1.3.6.1.4.1.95...
1 Mass-Training_P_00001_LEFT_MLO_1/1.3.6.1.4.1.9...
2 Mass-Training_P_00004_LEFT_CC_1/1.3.6.1.4.1.95...
3 Mass-Training_P_00004_LEFT_MLO_1/1.3.6.1.4.1.9...
4 Mass-Training_P_00004_RIGHT_MLO_1/1.3.6.1.4.1....

```

```

mask_path
0 Mass-Training_P_00001_LEFT_CC_1/1.3.6.1.4.1.95...
1 Mass-Training_P_00001_LEFT_MLO_1/1.3.6.1.4.1.9...
2 Mass-Training_P_00004_LEFT_CC_1/1.3.6.1.4.1.95...
3 Mass-Training_P_00004_LEFT_MLO_1/1.3.6.1.4.1.9...
4 Mass-Training_P_00004_RIGHT_MLO_1/1.3.6.1.4.1....
<class 'pandas.core.frame.DataFrame'>

```

	patient_id	breast_density	left or right	breast image view	abnormality id
0	P_00001	3	LEFT	CC	1
1	P_00001	3	LEFT	MLO	1
2	P_00004	3	LEFT	CC	1
3	P_00004	3	LEFT	MLO	1
4	P_00004	3	RIGHT	MLO	1

	abnormality type	mass shape	mass margins
0	mass IRREGULAR-ARCHITECTURAL_DISTORTION		SPICULATED
1	mass IRREGULAR-ARCHITECTURAL_DISTORTION		SPICULATED
2	mass ARCHITECTURAL_DISTORTION		ILL_DEFINED
3	mass ARCHITECTURAL_DISTORTION		ILL_DEFINED
4	mass	OVAL	CIRCUMSCRIBED

	assessment	pathology	subtlety
0	4	MALIGNANT	4
1	4	MALIGNANT	4
2	4	BENIGN	3
3	4	BENIGN	3
4	4	BENIGN	5

```

img_path \
0 Mass-Training_P_00001_LEFT_CC/1.3.6.1.4.1.9590...
1 Mass-Training_P_00001_LEFT_MLO/1.3.6.1.4.1.959...
2 Mass-Training_P_00004_LEFT_CC/1.3.6.1.4.1.9590...

```

```

3 Mass-Training_P_00004_LEFT_MLO/1.3.6.1.4.1.959...
4 Mass-Training_P_00004_RIGHT_MLO/1.3.6.1.4.1.95...

                                cropped_path \
0 Mass-Training_P_00001_LEFT_CC_1/1.3.6.1.4.1.95...
1 Mass-Training_P_00001_LEFT_MLO_1/1.3.6.1.4.1.9...
2 Mass-Training_P_00004_LEFT_CC_1/1.3.6.1.4.1.95...
3 Mass-Training_P_00004_LEFT_MLO_1/1.3.6.1.4.1.9...
4 Mass-Training_P_00004_RIGHT_MLO_1/1.3.6.1.4.1....

                                mask_path
0 Mass-Training_P_00001_LEFT_CC_1/1.3.6.1.4.1.95...
1 Mass-Training_P_00001_LEFT_MLO_1/1.3.6.1.4.1.9...
2 Mass-Training_P_00004_LEFT_CC_1/1.3.6.1.4.1.95...
3 Mass-Training_P_00004_LEFT_MLO_1/1.3.6.1.4.1.9...
4 Mass-Training_P_00004_RIGHT_MLO_1/1.3.6.1.4.1....

# Safety check: confirm full_mass is still the correct DataFrame
assert isinstance(full_mass, pd.DataFrame), "full_mass is not a DataFrame!"
assert "img_path" in full_mass.columns, "img_path column missing!"
assert "cropped_path" in full_mass.columns, "cropped_path column missing!"
assert "mask_path" in full_mass.columns, "mask_path column missing!"
assert "pathology" in full_mass.columns, "pathology column missing!"

# Convert relative paths to absolute JPEG paths
def safe_join(base_dir, path):
    if isinstance(path, str):
        return os.path.join(base_dir, path)
    return None

full_mass.columns = full_mass.columns.str.strip()

full_mass["img_path"] = full_mass["img_path"].apply(lambda x: safe_join(JPEG_DIR, x))
full_mass["cropped_path"] = full_mass["cropped_path"].apply(lambda x: safe_join(JPEG_DIR, x))
full_mass["mask_path"] = full_mass["mask_path"].apply(lambda x: safe_join(JPEG_DIR, x))

# Use cropped images for training
full_mass['img_path'] = full_mass['cropped_path']

# Filter only valid paths
full_mass = full_mass[full_mass['img_path'].apply(lambda x: isinstance(x, str) and os.path.exists(x))]
full_mass = full_mass[full_mass['mask_path'].apply(lambda x: isinstance(x, str) and os.path.exists(x))]
full_mass = full_mass.reset_index(drop=True)
full_mass

# Create label
class_mapper = {"MALIGNANT": 1, "BENIGN": 0, "BENIGN_WITHOUT_CALLBACK": 0}
full_mass['label'] = full_mass["pathology"].replace(class_mapper)

```



```
train_df = full_mass[['img_path', 'mask_path', "label"]]
```

```
-----
KeyError                                Traceback (most recent call last)
```

```
File /usr/local/lib/python3.8/site-packages/pandas/core/indexes/base.py:3653, in Index.get_loc
```

```
3652 try:
```

```
-> 3653     return self._engine.get_loc(casted_key)
```

```
3654 except KeyError as err:
```

```
File /usr/local/lib/python3.8/site-packages/pandas/_libs/index.pyx:147, in pandas._libs.index
```

```
File /usr/local/lib/python3.8/site-packages/pandas/_libs/index.pyx:176, in pandas._libs.index
```

```
File pandas/_libs/hashtable_class_helper.pxi:7080, in pandas._libs.hashtable.PyObjectHashTable
```

```
File pandas/_libs/hashtable_class_helper.pxi:7088, in pandas._libs.hashtable.PyObjectHashTable
```

```
KeyError: 'img_path'
```

The above exception was the direct cause of the following exception:

```
KeyError                                Traceback (most recent call last)
```

```
Cell In[29], line 9
```

```
5     return None
```

```
7 full_mass.columns = full_mass.columns.str.strip()
```

```
----> 9 full_mass["img_path"] = full_mass["img_path"].apply(lambda x: safe_join(JPEG_DIR, x))
```

```
10 full_mass["cropped_path"] = full_mass["cropped_path"].apply(lambda x: safe_join(JPEG_DIR, x))
```

```
11 full_mass["mask_path"] = full_mass["mask_path"].apply(lambda x: safe_join(JPEG_DIR, x))
```

```
File /usr/local/lib/python3.8/site-packages/pandas/core/frame.py:3761, in DataFrame.__getitem__
```

```
3759 if self.columns.nlevels > 1:
```

```
3760     return self._getitem_multilevel(key)
```

```
-> 3761 indexer = self.columns.get_loc(key)
```

```
3762 if is_integer(indexer):
```

```
3763     indexer = [indexer]
```

```
File /usr/local/lib/python3.8/site-packages/pandas/core/indexes/base.py:3655, in Index.get_loc
```

```
3653     return self._engine.get_loc(casted_key)
```

```
3654 except KeyError as err:
```

```
-> 3655     raise KeyError(key) from err
```

```
3656 except TypeError:
```

```
3657     # If we have a listlike key, _check_indexing_error will raise
```

```
3658     # InvalidIndexError. Otherwise we fall through and re-raise
```

```
3659     # the TypeError.
```

```
3660     self._check_indexing_error(key)
```

```

KeyError: 'img_path'

print(full_mass.columns.tolist())

train_tf = transforms.Compose([
    transforms.ToPILImage(),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor()
])

val_tf = transforms.Compose([
    transforms.ToPILImage(),
    transforms.ToTensor()
])

full_dataset = CBISDataset(train_df, transform=train_tf)

val_ratio = 0.2
val_size = int(len(full_dataset) * val_ratio)
train_size = len(full_dataset) - val_size

train_ds, val_ds = random_split(full_dataset, [train_size, val_size])
val_ds.dataset.transform = val_tf

train_loader = DataLoader(train_ds, batch_size=16, shuffle=True, num_workers=2)
val_loader = DataLoader(val_ds, batch_size=16, shuffle=False, num_workers=2)

```

Model, optimizer, scheduler, early stopping, checkpointing

```

from torchvision import models
import torch.nn as nn
import torch.optim as optim

resnet = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
num_features = resnet.fc.in_features
resnet.fc = nn.Sequential(nn.Dropout(0.5), nn.Linear(num_features, 1))
resnet = resnet.to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(resnet.parameters(), lr=1e-4, weight_decay=1e-5)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min", factor=0.5, patience=10)

print(train_df["mask_path"].head())
print(train_df["mask_path"].apply(type).value_counts())

```

Training loop (prototype: ~5–10 epochs)

```
# %%  
# Training loop (prototype: ~5-10 epochs)  
  
from tqdm.auto import tqdm  
  
def train_one_epoch(model, loader, optimizer, criterion, device):  
    model.train()  
    running_loss = 0.0  
  
    for imgs, masks, labels in tqdm(loader, desc="Train", leave=False):  
        imgs = imgs.to(device)  
        labels = labels.float().to(device)  
  
        optimizer.zero_grad()  
        outputs = model(imgs).squeeze(1)  
        loss = criterion(outputs, labels)  
        loss.backward()  
        optimizer.step()  
  
        running_loss += loss.item() * imgs.size(0)  
  
    return running_loss / len(loader.dataset)  
  
def evaluate(model, loader, criterion, device):  
    model.eval()  
    running_loss = 0.0  
    all_outputs = []  
    all_labels = []  
  
    with torch.no_grad():  
        for imgs, masks, labels in tqdm(loader, desc="Val", leave=False):  
            imgs = imgs.to(device)  
            labels = labels.float().to(device)  
  
            outputs = model(imgs).squeeze(1)  
            loss = criterion(outputs, labels)  
  
            running_loss += loss.item() * imgs.size(0)  
            all_outputs.append(outputs.cpu())  
            all_labels.append(labels.cpu())  
  
    all_outputs = torch.cat(all_outputs)  
    all_labels = torch.cat(all_labels)
```

```

probs = torch.sigmoid(all_outputs)
preds = (probs > 0.5).int()
acc = (preds == all_labels.int()).float().mean().item()

return running_loss / len(loader.dataset), acc

EPOCHS = 10
for epoch in range(EPOCHS):
    train_loss = train_one_epoch(resnet, train_loader, optimizer, criterion, device)
    val_loss, val_acc = evaluate(resnet, val_loader, criterion, device)
    print(f"Epoch {epoch+1}/{EPOCHS} | Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f}")

```